iRDRC: Markov Decision Process Deep Dive & Implementation

BTP Discussion 2

Joel James & Majida Bee
29 August 2026

Focus: Complete mathematical formulation of the MDP, why it works, and how to implement it.

Recap: The iRDRC Problem

Aspect	Description
Problem	AV needs radar + communication on shared resource
Environment	Weather, road, speed, moving objects (binary states)
Objective	Maximize throughput while minimizing miss-detection risk
Approach	Adaptive learned policy via Deep Q-Network (DQN)
Key Insight	Fixed scheduling fails under dynamic uncertainty

Meeting 1 Covered: System model (queue, channel, radar), environment model (Eq. 1), and introduced MDP components. **Today's Focus:** Deep understanding of MDP formulation, why each component matters, and how DQN solves it.

Why Markov Decision Process (MDP)?

The Markov Property: The future depends only on the current state, not on history.

Why this matters for iRDRC:

1. Queue state *d* captures the current backlog; only *d* matters for next queue size, not when packets arrived

2. Channel state *c* is independent each step; history doesn't help predict next *c*

3. Environmental factors (r, w, v, m) are resampled; each time step is a fresh draw

4. Radar detection is perfect; if radar is on and event exists, we catch it regardless of past misses

Result: Given current state *s* and action *a*, the next state *s'* is probabilistically determined, allowing us to use RL algorithms that exploit the Markov property.

The MDP Tuple: ■S, A, P, R, γ■

A Markov Decision Process is defined by five components. Together, they describe a complete sequential decision problem:

Component	Symbol	Meaning (in iRDRC)	Example
State Space	S	All possible observations the AV can make	(d=5, c=0, r=0, w=1, v=1, m=0)
Action Space	A	All possible choices the AV can make	{0: communicate, 1: radar}
Transition Prob.	P(s′|s,a)	How state evolves given action	P(d=4 | d=5, a=0, c=0)
Reward Function	R(s,a,s′)	Immediate reward for (state, action, next state)	R = +2 (good comm), -50 (missed event)
Discount Factor	γ	Weight on future vs. immediate rewards	γ = 0.99

State Space (Equation 2)

Definition: $S = \{ (d, c, r, w, v, m) : d \in \{0,\ldots,10\}, c,r,w,v,m \in \{0,1\} \}$

State Space Size Calculation:

• d (queue): 11 possible values (0, 1, 2, ..., 10)

• c (channel): 2 possible values (0 = good, 1 = bad)

• r (road): 2 possible values

• w (weather): 2 possible values

• v (speed): 2 possible values

• m (moving objects): 2 possible values

Total: 11 × 2■ = 352 states

Key Insight: 352 states is manageable but large enough to benefit from function approximation (i.e., neural networks) instead of tabular methods.

Action Space (Simple but Powerful)

Definition: A = {0, 1}

What Each Action Means:

Action	Mode	What Happens	Performance Impact
a = 0	Communication	Transmit queued packets; channel determines success	Increases throughput, increases miss-detection risk
a = 1	Radar	Scan for obstacles; if event exists and radar is on, it is detected	Decreases throughput, decreases miss-detection risk

The Central Design Decision: Each time step, the AV chooses exactly one mode. This binary action is the lever that the learning algorithm will adjust to find the optimal balance.

Reward Function (Equation 3) — The Learning Signal

The reward function translates engineering objectives (throughput + safety) into numbers that a learning algorithm can optimize. It is the bridge between goals and learning.

Action	Situation	Reward	Interpretation
Comm (a=0)	Good channel, no event	+r■ = +2	Success: 4 packets sent
Comm (a=0)	Bad channel, no event	+r■ = +1	Partial: 2 packets sent
Comm (a=0)	Unexpected event occurs	–r■ = –50	Catastrophic: missed obstacle
Radar (a=1)	No event occurs	0	Radar cost: detection found nothing
Radar (a=1)	Event occurs (b unfav.)	+r■(b+1) = +5(b+1)	Success: detected threat reward scales with risk

Key Design Choices:

• –**50 ■ r■, r■ (Safety-First):** Missing an event is 25–50× worse than missing a few packets. This asymmetry encodes a safety priority.

• **Reward scales with risk (b):** A detected event in high-risk (b=4) earns +25, but in low-risk (b=0) earns only +5. Encourages the policy to patrol risky conditions.

• **Radar with no event = 0:** No reward, but no penalty either. Radar is insurance against uncertainty.

Objective: Discounted Return (Equation 4)

Goal: Find a policy π that maximizes the expected cumulative reward over many time steps.

Discounted Return (Value of a Policy π):
$G(\pi) = E\{ \Sigma_{t=0}^{T} \gamma^t r_{t+1}(\pi) \}$

Components:

• $r_{t+1}(\pi)$: Immediate reward at time t+1 under policy π

• $\gamma \in (0,1)$: Discount factor (paper doesn't specify; typical ≈ 0.99)

• γ^t: Exponential decay; rewards 10 steps away are worth $\gamma^{10} \approx 0.90\times$ compared to immediate

• T: Time horizon (episode length; likely 500–1000 steps)

Optimal Policy: $\pi^* = \arg\max_{\pi} G(\pi)$
The policy that, when followed, yields the highest expected cumulative reward.

Intuition: Don't chase immediate rewards. Choose actions that lead to long-term payoff. If communication now leads to a crash later, the –50 penalty far outweighs the +1 or +2 gain.

Why Discount the Future? (γ Explained)

What γ Controls:

• γ → *1.0 (e.g., 0.99):* Future rewards nearly as valuable as immediate rewards. Policy is far-sighted. Over time horizon T=1000, reward at step 999 is still worth γ■■■ ≈ 0.0005 of immediate reward.

• γ → *0.0 (e.g., 0.5):* Only immediate rewards matter. Policy is myopic; will choose any immediate gain even if it ruins future prospects.

For iRDRC:

• γ ≈ 0.99 makes sense because a crash today has consequences for all future steps.

• If γ were 0.5, the policy might communicate recklessly now to gain +2, knowing the –50 crash penalty is only 50% as important.

• γ ≈ 0.99 ensures the policy balances immediate throughput against long-term safety.

From MDP to Policy: Q-Values and π

Action-Value Function Q(s, a): The expected discounted return starting from state *s*, taking action *a*, and then following the optimal policy forever.

$Q^*(s, a) = E\{ r + \gamma \max_{a'} Q^*(s', a') \}$

Intuition:

• Q(d=0, a=0) = high if the queue is empty and communication is valuable

• Q(d=10, a=0) = lower if the queue is full and can't hold more packets

• Q(w=1, v=1, a=0) = very low if weather and speed are unfavorable (high event probability) and we choose communication

• Q(w=1, v=1, a=1) = higher because radar protects us from the high-risk state

Optimal Policy π^*: $\pi^*(s) = \arg\max_{a \in A} Q^*(s, a)$
At each state, pick the action with the highest Q-value.

Challenge: We don't know Q*(s, a) in advance. We must learn it from experience. That's where Q-learning and DQN come in.

Environment Dynamics: State Transitions P(s'|s,a)

Queue Dynamics (Deterministic):

$d'_{t+1} = \min(d_t - (\text{transmission\_count if } a_t=0) + \text{arrivals}, D)$

Where transmission_count depends on channel state: $\nu_{\blacksquare}=4$ if c=0 (good), $\nu_{\blacksquare}=2$ if c=1 (bad). Arrivals ~ Poisson(λ_d=1).

Channel Dynamics (Stochastic, Independent):

c'_{t+1} ~ Bernoulli(p_c=0.1) $\rightarrow$ P(c'=1 | c, a) = 0.1 (independent of c and a)

Environmental Factors (Stochastic, Independent):

Each of r, w, v, m is resampled each step: P(x'=1) = 1 – τ_x. All four factors are independent of each other and of past history (Markov property).

Unexpected Event Generation (Stochastic, Deterministic given state):

$\oplus_t$ ~ Bernoulli(P($\oplus$)) where P($\oplus$) is computed via Eq. (1). If a_t=1 (radar) and $\oplus_t$=1, we detect it. If a_t=0 (communication) and $\oplus_t$=1, we miss it (–50 penalty).

Tabular Q-Learning vs. Deep Q-Network (DQN)

Aspect	Tabular Q-Learning	Deep Q-Network (DQN
Storage	352 table entries (one per state-action pair)	Neural network weights
Update Rule	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$	Same TD target but applied to network output
Function Approx.	None; exact values	Neural network approximates Q(s,a) for any s
Convergence	~280 episodes (paper results)	~170 episodes (paper results)
Scalability	352 states OK; larger problems intractable	Scales to larger state spaces (feature learning)
Training Data Efficiency	One update per experience	Mini-batch updates from replay memory (higher variance reduction)
Exploration	ε-greedy (simple)	ε-greedy (simple)

Bottom Line: For iRDRC with 352 states, both work. DQN is faster and scales better. Both methods exploit the MDP structure: knowing P(s'|s,a) is unnecessary; learning from experience is enough.

DQN Network Update (Equation 5)

Update Rule:

$$\theta_{t+1} = \theta_t + \alpha \left[y_t - Q(s_t, a_t; \theta_t) \right] \nabla Q(s_t, a_t, \theta_t)$$

Where:

- θ_t: Network weights at time t
- α: Learning rate (e.g., 0.001)
- y_t: Target = $r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-_t)$
- θ^-_t: **Target network** weights (copied from online network periodically, e.g., every 10k steps)
- $\nabla Q(...)$: Gradient of Q-value w.r.t. weights

Key Innovation: Target Network

Why use two networks? The target network provides a stable target for learning. Without it, y_t changes as θ changes, causing oscillations. By updating θ^- only periodically, we stabilize training.

DQN Training Loop: 7 Steps per Experience

Step	Action	Details
1	Observe state	AV measures (d, c, r, w, v, m)
2	Estimate Q-values	Feed state to network → get Q(s, 0) and Q(s, 1)
3	Choose action	ε-greedy: explore (random a) with prob ε, exploit (max Q) with prob 1–ε
4	Interact	Execute action a; environment returns reward r and next state s'
5	Store in memory	Add (s, a, r, s') to replay buffer (circular buffer, capacity ~10k)
6	Sample and train	Random mini-batch (size ~32) from buffer; compute loss and update θ
7	Repeat	Continue for many episodes until cumulative reward converges

Why Replay Memory? Consecutive experiences are highly correlated (e.g., queue decreases only slightly each step). Sampling random mini-batches decorrelates the data, reducing variance and stabilizing learning.

Summary: Complete MDP for iRDRC

Component	Definition	Size/Form	Implementation Note
State Space S	(d, c, r, w, v, m)	352 discrete states	Fully observable; measured sensors
Action Space A	{0: comm, 1: radar}	2 actions	Binary choice each step
Reward R(s,a,s′)	r■=2, r■=1, r■=–50, r■=–5, (–50 to +25	Scalar	Safety-first asymmetry
Transition P(s'|s,a)	Queue (det.); channel, env (stoch.)	Bernoulli(q) event prob.	Markov: future indep. of history
Discount γ	Typical ≈ 0.99	Scalar in [0,1)	Not specified in paper; choose typical value
Solver	DQN (Keras/PyTorch)	Neural net (6→hidden→2)	~720 episodes to converge

Key Takeaways: Why MDP + DQN Works

1. MDP Captures the Structure: The Markov property holds; past doesn't matter, only current state. This allows algorithms to plan without knowing transition probabilities.

2. Reward Design Encodes Goals: The asymmetric reward (–50 ■ +1,+2) directly implements a safety priority. Learning doesn't require explicit rules; it discovers the right balance.

3. Discount Factor Ensures Long-Horizon Planning: $\gamma \approx 0.99$ makes the policy care about future crashes. Without it, the policy would be greedy and reckless.

4. DQN Scales Beyond Tabular Methods: 352 states is manageable for both Q-learning and DQN. DQN is faster (170 vs. 280 episodes) due to mini-batch updates and replay memory stabilization.

5. Adaptation Emerges from Learning: The policy is not programmed to 'use radar when risky.' It learns this strategy by optimizing cumulative reward, adapting to any change in p^1■ or other conditions.

Modeling Gaps to Address Before Implementation

Gap	What's Missing	Action for Implementation
Probability Values	p■■, p¹■, p■■, p¹■ not specified	Choose reasonable estimates (e.g., p■=0.01, p¹=0.05)
State Evolution	τ■ (probability of favorable state) not given	Assume τ■ ≈ 0.7, τ_w ≈ 0.8, τ■ ≈ 0.6, τ_m ≈ 0.7
Discount Factor γ	Not specified in paper	Use γ ≈ 0.99 (standard choice)
Episode Length T	Not explicit; appears to be 500–1000 steps	Experiment with T=500, measure convergence
DQN Hyperparams	Learning rate α, batch size, buffer size, and decay schedules	Use standard DQN defaults: α=0.001, batch=32, buffer=10k, ε-decay li
Network Architecture	Number of hidden layers and units not specified	Try 2 hidden layers, 64 units each (6→64→64→2)

Ready for Implementation

✓ MDP structure fully understood: state, action, reward, transition, objective

✓ Why each component matters: Markov property, safety-first reward, discount factor

✓ DQN algorithm explained: network update, replay memory, target network

✓ Gaps identified and solutions proposed

Next Meeting (Discussion 3): Implementation code walkthrough and first results

Questions & Discussion